

第1章 Linux 操作系统概述

本章学习目标

知识目标

- 了解Linux操作系统起源、发展过程
- 掌握Linux操作系统的特点
- 掌握Linux内核源代码的组织结构
- 掌握Linux内核分析方法

能力目标

- 能够认识Linux操作系统版本的含义
- 能够分析与其他产品的区别

素质目标

- 增强思考观察能力
- 强化分析比较能力

第1章 Linux 操作系统概述

1. Linux的起源与发展
2. 自由软件GNU与GPL
3. Linux的特点
4. Linux系统及内核的组成
5. Linux版本的识别
6. Linux内核源代码的组织
7. Linux内核分析方法
8. 与其它几种操作系统的比较
9. Linux中的C语言及汇编语言

1、Linux的起源与发展

- 1990年秋天，学生Linux在芬兰首都赫尔辛基大学学习操作系统课程。因为上机需要排队等待，Linux买了台PC机，开发了第一个程序。程序包括两个进程，向屏幕上写字母，然后用定时器来切换进程
- Linux需要终端仿真程序来存取Usenet新闻组的内容，于是他写了从调制解调器上收发信息的程序以及显示器、键盘和调制解调器的驱动程序
- 然后，写了磁盘驱动程序，文件系统
- 一旦有了进程切换、文件系统和设备驱动程序，就拥有了一个操作系统原型，或者至少是它的一个内核
- Linux就以这样极其古怪但也极其自然式问世

1、Linux的起源与发展

Linux之父-Linus Torvalds



- 芬兰赫尔辛基大学:1990
- Linux是一匹自由而奔放的黑马
 - 诞生于学生之手
 - 成长于Internet
 - 壮大于自由而开放的文化



1、Linux的起源与发展

Linux内核编年史

- 1990年: Linux 0.0.1
- 1991年: Linux 0.0.2 bash, gcc
- 1993年底: Linux 1.0版诞生
- 1994年: Linux 的第一个商业发行版 Slackware 问世
- 1996年: 美国国家标准技术局的计算机系统实验室确认Linux 版本 1.2.13 (由 Open Linux 公司打包)，符合 POSIX 标准
- 1999年: Linux 2.2发布，64位平台，SMP，NTFS，SCSI，多媒体
- 2001年: Linux 2.4版内核发布
- 2003年: Linux 2.6版内核发布
-
- 2020年12月: Linux 5.10版内核发布
-
- 2022年10月: Linux 6.0.10版内核发布

2、自由软件与GPL

- 自由软件: 指允许自由地使用、复制、修改、分发的软件，软件的源代码必须是可得到的。
 - GNU/Linux、freeBSD
 - 电子邮件服务器 sendmail
 - 服务器 Apache
- 自由软件基金会FSF (Free Software Foundation)
- 自由软件之父: Richard Stallman
- Linux内核的开发工作都是基于GNU推出的自由软件完成的。GNU和Linux相辅相成。

2、自由软件与GPL

• POSIX

- UNIX发展过程中，出现了许多变种 (BSD, System V等)。
- 1986年，IEEE的一个委员会指定了一套“基于Unix的可移植操作系统”的标准，命名为**POSIX标准 (Portable Operating System Interface Standard)**
- POSIX定义了一套标准的操作系统接口和工具，并由ISO接受为国际标准。几乎所有的正式操作系统都以支持POSIX标准为目标。易移植。

2、自由软件与GPL

◆GNU项目

- 开始于1984年，由Richard Stallman (自由软件之父) 创立，主要由自由软件基金 (FSF) 资助的一个项目。
- 目标: 是开发一个完全自由的、UNIX类型的操作系统，称为GNU系统。与此相应的有一份通用公共许可证GPL。
- GNU软件的开发不是为了经济目的，而是不断开发并传播新的软件，每个人都能自由获得。

2、自由软件与GPL

◆通用公共许可证(General public License)

- 保证自由软件对所有用户都是自由的(不一定免费)
- 所有的GNU软件和派生工作均遵循GNU通用公共许可证，即GPL。http://www.gnu.org/licenses/licenses.html
- ◆GPL类软件遵循一定的规则
 - 传播者不能限制购买软件的用户自由权。如果用户买了一套GPL软件，就可免费复制、传播或出售。
 - 传播者必须清楚告诉用户该软件属于GPL软件。
 - 传播者必须免费提供软件的完整源代码，且任何用户能以源代码形式将软件复制或发布给别用户。
 - 如果用户的软件使用了受GPL保护的软件的一部分，则该软件也成为GPL软件，它必须随应用程序一起发布源代码。

2、自由软件与GPL

■ GNU/Linux

■ Linux:内核

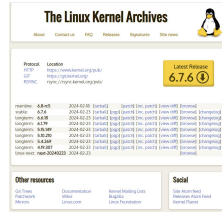
■ GNU/Linux: 操作系统名称。

- 使Linux内核变得强大的大量应用程序都是GNU软件，都是在GPL许可证下发布的。如：窗口管理工具、编译器、各种shell、浏览器等。

2、自由软件与GPL

<http://www.kernel.org/>

Linux内核维护网站



3、Linux系统的主要特点

- Linux是一套免费使用的，遵循 GPL、POSIX标准的、与UNIX兼容的操作系统。
- 能够在从微型计算机到大型计算机的任何环境中在任何平台上运行，良好的可移植性。
- 可运行于多种平台（x86、Alpha、MIPS、PowerPC、SPARC等）之上，是目前支持硬件最多的操作系统
- 拥有现代操作系统具有的所有内容，例如
 - 抢占式多任务处理，支持多用户
 - 内存保护
 - 支持SMP，支持TCP/IP
 - 支持绝大多数的32位和64位CPU 等
- 还有一些其他操作系统没有的特色，比如：
 - NFS，VFS，高效的EXT系列文件系统等等

3、Linux系统的主要特点

◆ 广泛的支持

- 处理器芯片的广泛支持：Intel x86, SPARC, PowerPC, Alpha, IA64
- 硬件公司：IBM, HP, Sun, Intel, AMD, Sony
- 软件公司：Oracle, Informix, Ingres, SAP, Borland等
- 对即插即用设备的支持
- 广泛的文件系统的支持：VFAT, FAT, HFS, HPFS, NTFS, Ext2等
- 大型数据库公司的支持：Oracle, Informix, Ingres

3、Linux系统的主要特点

• 企业应用领域

- 成熟的服务器级的操作系统、大型文件系统、多处理器、日志型文件系统、高可用集群、提供web服务等

• 嵌入式领域

- 商业化的嵌入式操作系统：VxWorks, pSOS, Windows CE等：高价格，源代码封闭
- Linux在嵌入式系统蓬勃发展，广泛应用
 - 路由器，防火墙，机顶盒，电器设备

3、Linux系统的主要特点

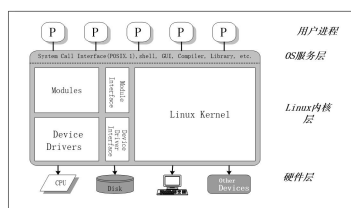
- 设备独立性：把所有外部设备统一当作文件来看待，只要安装它们的驱动程序，用户可以象使用文件一样使用设备
- 丰富的网络功能：具有完善的内置网络，大量有用的实用程序和通信、联网工具，可以实现远程管理功能
- 性能高和安全性强：Linux采取了许多安全技术措施，包括对读、写控制、带保护的子系统、审计跟踪、核心授权等，可以作为中小型ISP或Web服务器工作平台。
- 便于定制和再开发：在遵从GPL版权协议的条件下，各部门、企业、单位或个人可根据自己的实际需要和使用环境对Linux系统进行裁剪、扩充、修改或者再开发。

3、Linux系统的主要特点

• 一个杰出的学习系统

- 很多顶级的系统开发平台，友好的编程语言及编程工具都免费包含在操作系统里面
- 病毒入侵，计算机的“后门”，软件提供商的“特别要求”，强迫性的软件升级，专有的文件格式，软件使用许可证和市场策略，产品注册登记，高得惊人的软件价格，等等……Linux都没有

4. Linux 系统及内核的组成

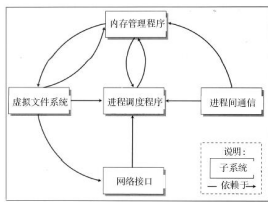


4. Linux 系统及内核的组成

◆ Linux 系统由四部分组成

- 用户进程：当一个用户程序在操作系统之上运行时，它成为操作系统中的一个进程。
- 系统调用接口(OS服务层)：在应用程序中，可通过系统调用来调用操作系统内核中特定的过程，以实现特定的服务。
- Linux内核：内核是操作系统的灵魂，它负责管理磁盘上的文件、内存，负责启动并运行程序，负责从网络上接收和发送数据包等等。
- 硬件：...

4. Linux 系统及内核的组成



Linux 内核的组成

4. Linux 系统及内核的组成

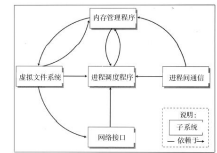
Linux 内核由4部分组成:

- **进程调度程序**: 负责控制进程访问CPU。确保所有的进程都能公平访问CPU。
- **内存管理程序**: 使多个进程可以安全地共享机器的主存系统, 并支持虚拟内存。
- **虚拟文件系统**: 提供一个所有设备的公共文件接口, 抽象了不同硬件设备的细节。
- **网络接口**: 提供对多种网络协议标准和网络设备的访问。
- **进程间通信**:

4. Linux 系统及内核的组成

◆ 分析:

- 进程调度与内存管理之间的关系
- 进程间通信与内存管理的关系
- 虚拟文件系统与网络接口之间的关系
- 内存管理与虚拟文件系统之间的关系



Linux内核各个子系统之间的依赖关系

4. Linux 系统及内核的组成

Linux内核的特征如下:

(1) **使用单一内核结构**。但是, 与传统的UNIX的单一内核结构不同的是, Linux可以动态装入和卸载内核中的部分代码。Linux将这些代码段称为模块(module), 并给予强有力的支持, 可以根据需要自动装入和卸载。

也就是说, Linux内核运行在单独的内存地址空间。Linux汲取了微内核的精华: 模块化设计、抢占式内核、支持内核线程以及动态装载内核模块的能力。Linux还避其微内核设计上性能损失的缺陷, 让所有事情都运行在内核态, 直接调用函数, 无需消息传递。

4. Linux 系统及内核的组成

- (2) **进程调度方式简单而有效**。对于用户进程, Linux采用简单的动态优先级调度方式; 对于内核中的例程如设备驱动程序、中断服务程序等采用软中断机制, 保证了内核例程的高效运行。
- (3) **动态加载内核模块**。尽管Linux内核也是单内核, 可是允许在需要的时候动态地卸载和加载部分内核代码。模块机制使内核保持独立而又易于扩充, 如可以裁减得非常小适合于嵌入式系统。
- (4) **支持对称多处理机制**。尽管许多Unix的变体也支持SMP, 但传统的Unix并不支持这种机制。
- (5) **Linux内核可以抢占**。与传统的Unix不同, Linux内核具有允许在内核运行的任务优先执行的能力。在其他各种Unix产品中, 只有Solaris和IRIX支持抢占, 但是大多数传统的Unix内核不支持抢占。

4. Linux 系统及内核的组成

(6) **Linux对线程的支持**: 内核并不区分线程和其他的一般进程。对于内核来说, 所有的进程都一样—只不过其中的一些共享资源而已。

Linux提供具有设备类的面向对象的设备模型、热插拔事件, 以及用户空间的设备文件系统。

5. Linux版本认识

- **版本**: 分为内核版、发行版
 - **内核版**: 分为稳定版、测试版
 - **发行版**: 与基本组成与安装、配置、技术支持、第三方应用有关
 - 发行版=内核+tools
- **Linux内核采用的是双树系统**
 - 一棵是**稳定树**, 主要用于发行, 新推出的稳定内核大部分都只是修正了一些bug或是加入了一些新的设备驱动程序。
 - 另一棵是非稳定树或称为**开发树**, 用于产品开发和改进, 变化很快。

5. Linux版本认识

Linux内核版本号由3位数字组成x.y.z

- 使用三个或者四个用“.”分隔的数字来代表不同内核版本。
- 第一个数字是主版本号, 第二个数字是从版本号, 第三个数字是修订版本号, 第四个可选的数字为稳定版本号 (stable version)。
- 从版本号可以反映出该内核是一个稳定版本还是一个处于开发中的版本, 该数字如果是偶数, 那么此内核就是稳定版; 如果是奇数, 那么它就是开发版。
- 例如: 版本号为2.6.30.1的内核——稳定版。内核的主版本号是2, 从版本号是6, 修订版本号是30, 稳定版本号是1。
- 头两个数字在一起描述了“内核系列”——在这个例子中, 就是2.6版内核系列。

5. Linux版本认识

◆ 例:



◆ 显示内核信息

```
#uname -r 内核版本号
#uname -s 内核名称
#uname -v 内核版本发布时间
#uname -a 所有信息
```

5、Linux版本认识

- 最近几年，每一个2.6系列的修订版本（比如2.6.29）都会较其前身有深刻的变化，也都会经历数月才面世。它更有利于在引入新特性的同时，维持系统的稳定性。
- 为了解决版本发布周期变长的副作用，内核开发者们引入了**稳定版本号**。
- 稳定版本号（如2.6.32.8中的8）包含了关键性bug的修改，并且常会向前移植处于开发版内核（如2.6.33）的重要修改。依靠这种方式，以前版本保证了仍然能将重点放在稳定性上。

5、Linux版本认识

- 发行版本 (Distribution)**
几种流行的Linux发行版本



5、Linux版本认识



5、Linux版本认识

发行版基本组成

- 内核**: 系统核心、网络堆栈
- 引导管理器**: MBR、LILO
- 用户界面**: 命令行、图形 (CDE/KDE/GNOME)
- X Window**: 图形子系统, Xfree86
- 网络**: DNS/HTTP/SMTP/FTP/NNTP/PPP
- 文件及打印服务**: NFS、Samba
- 数据库**: mysql、PostgreSQL、DB2、Oracle 8、Sybase
- 应用程序**: Netscape/StarOffice/xIRC/LICQ/Gimp
- 包管理器**: 安装/反安装/升级应用程序、DPM / RPM
- 开发工具与库**: gcc、gdb、glibc、libc

6、Linux内核源代码的组织

- 理解Linux源码的布局规则**
 - 指导Linux移植, 添加驱动程序时需要修改哪些目录或者文件
- 如何得到源代码?**
 - 在Linux系统根目录下, usr/src/linux目录下就是内核源代码。可以看到大量子目录。按树形结构组织。

6、Linux内核源代码的组织

- arch**: 包含了所有体系结构相关的核心代码。它还包含每种支持的体系结构的子目录, 如i386。每个体系结构子目录下包含几个主要的子目录:
 - kernel**: 与体系结构相关的**核心代码**
 - mm**: 与体系结构相关的**内存管理代码**
 - lib**: 与体系结构相关的**库代码**
- include**: 包括了用来编译内核的大多数头文件。对于每种支持的体系结构分别有一个子目录。
- init**: 包含内核的初始化代码, 内核开始工作的起点。
- mm**: 包含独立于CPU体系结构的所有的内存管理代码。

6、Linux内核源代码的组织

- modules**: 仅仅包含已建好的模块。
- fs**: 所有的文件系统代码。它被划分成对应不同文件系统的子目录, 如vfat和ext2。
- kernel**: 存放进程管理和调度等的主要核心代码, 用于实现系统的核心模块, 如: 进程管理、进程调度、中断处理、时钟管理、同步机制等。如fork.c, sched.c, time.c
- net**: 核心的网络部分代码。如各种网络协议, 以及网卡驱动程序
- lib**: 此目录包含了核心的库代码。与处理器结构相关库代码被放在arch/lib/目录下。
- scripts**: 此目录包含用于编译内核时用的脚本文件 (如mk和tk脚本)
- drivers**: 系统中所有的设备驱动都位于该目录中。它又进一步划分成几类设备驱动, 如block。
- ipc**: 包含了内核的进程间通讯代码。

6、Linux内核源代码的组织

- 每个目录下都有一个depend文件和makefile文件, 是编译时使用的辅助文件。
 - 仔细阅读这两个文件, 对搞清楚各个文件之间的联系和依托关系很有帮助。
- Linux内核源代码**
 - 放置在/usr/src/linux目录下
 - 对内核的修改是用补丁文件的方式发布的, patch实用程序用来对内核源文件进行一系列修改。
 - 例: 现有内核 2.2.xx, 想升级到2.4.18, 则需要先获得2.4.18的补丁文件, 应用patch来修改2.2.xx文件。
 - ```
cd /usr/src/linux
patch -p1 < patch-2.4.18
```

## 6、Linux内核源代码的组织

### Linux内核对SMP的支持

- 每个处理器都在运行着进程, 当运行在某个处理器上的进程耗尽其时间片或该进程处于等待状态时, 该处理器将单独运行调度程序来选择新的进程。
- 每个处理器都有自己的空闲进程, 都有自己的当前进程。
  - Task\_struct结构中包含:
    - Processor字段, 当前运行该进程的处理器编号
    - Last\_processor字段, 上次运行该进程的处理器编号
    - Processor\_musk字段, 该字段第N位为1, 表示该进程可以运行在第N个处理器上。利用该字段可以将某个进程限制在单个处理器上运行。

6、Linux内核源代码的组织

◆在arch/i386下存在如下目录

|                      |          |                |
|----------------------|----------|----------------|
| boot                 | 文件夹      | 2007-6-4 17:36 |
| i386的启动源代码文件目录       | 文件夹      | 2007-6-4 17:36 |
| i386的核心源代码文件目录       | 文件夹      | 2007-6-4 17:36 |
| i386的库源代码文件目录        | 文件夹      | 2007-6-4 17:36 |
| i386的数字仿真源代码文件目录     | 文件夹      | 2007-6-4 17:36 |
| i386的内存管理源代码文件目录     | 文件夹      | 2007-6-4 17:36 |
| defconfig            | 19 KB 文件 | 2002-2-26 3:37 |
| i386的配置文件            | 4 KB 文件  | 2001-4-13 3:20 |
| i386的体系相关部分的Makefile | 35 文件    | 2002-2-26 3:37 |
| i386的Linux内核的链接描述文件  |          |                |

考虑arch为i386

6、Linux内核源代码的组织

◆在arch/i386/boot目录下

|            |                            |            |
|------------|----------------------------|------------|
| compressed | i386的体系结构相关部分的启动代码都采用汇编码写的 | 类          |
| tools      |                            | 文件夹        |
| bootsect.S |                            | 11 KB S 文件 |
| in         | 启动扇区中的启动代码                 | 1 KB SH 文件 |
| Ma         | 其目标码必然是512字节               | 3 KB 文件    |
| setup.S    |                            | 26 KB S 文件 |
| vi         | i386初始化                    | 39 KB S 文件 |
| head.S     |                            | 3 KB S 文件  |
| misc       | i386的内核解压缩                 | 9 KB C 文件  |

i386的启动代码文件

6、Linux内核源代码的组织

◆在arch/i386/kernel目录下的.S文件

|              |            |
|--------------|------------|
| trampoline.S | 2 KB S 文件  |
| head.S       | 11 KB S 文件 |
| entry.S      | 18 KB S 文件 |

main 这是体系结构无关部分，i386体系结构相关部分的启动，其目的就是进入main.c中的start\_kernel处执行

i386的启动代码文件

6、Linux内核源代码的组织

Linux内核源代码的几大组成部分

- 1、系统启动与初始化
- 2、内存管理
- 3、内核
- 4、进程间通信
- 5、中断处理
- 6、设备驱动程序
- 7、文件系统
- 8、网络
- 9、模块

7、Linux内核分析方法

◆如何分析内核源代码

- 由于内核代码冗长、内核体系结构的庞杂，所以分析内核也是一个很艰难、很需要毅力的事
  - 弄懂源码的文件组织形式
  - 以程序流程为线索，一线串珠
  - 以数据结构为基点，触类旁通
  - 以功能为中心，各个击破

◆分析思路

- 概念和数据结构
- 对外接口
- 模块内部处理过程：算法

◆本课程以什么样的版本进行研究？

- Linux2.4.18+Linux 2.6的新增功能

7、Linux内核分析方法

| 后缀名       | 文件类型             |
|-----------|------------------|
| .c        | C语言源代码文件         |
| .c或cc或cxx | C++源代码文件         |
| .m        | Objective C源代码文件 |
| .s        | 汇编语言源代码文件        |
| .h        | 程序所包含的头文件        |
| .i        | 已预处理过的C源代码文件     |
| .ii       | 已预处理过的C++源代码文件   |
| .S        | 已预编译的汇编语言源代码文件   |
| .o        | 编译后的目标文件         |
| .so       | 目标文件构成的动态库文件     |
| .a        | 目标文件构成的静态库文件     |

Linux程序文件的后缀名

7、Linux内核分析方法

分析和实验验证环境-VMware

- ◆模拟x86硬件平台
- ◆可以在Windows、Linux等平台上运行
- ◆VMWare模拟出来的硬件包括：
  - 主板、内存、硬盘（IDE和SCSI）、DVD/CD-ROM、软驱、网卡、声卡、串口、并口和USB口。
- ◆可以直接从ISO文件上安装
- ◆在一个VMware上安装的的操作系统形成的虚拟机，可以拷贝到其他装有VMware的机器上

7、Linux内核分析方法

内核分析工具

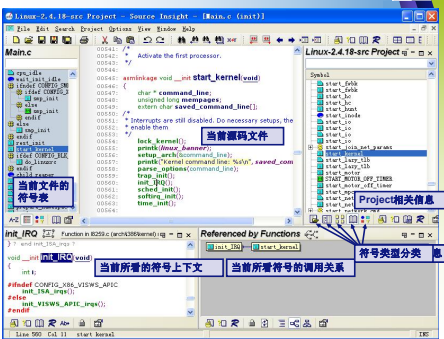
- ◆Source Insight: Windows平台上的
- ◆LXR (Linux Cross Reference): Linux代码交叉检索工具，Linux平台上的，通过对符号建立索引，用户界面是浏览器<http://lxr.Linux.org>
- ◆Kscope: <http://kscope.sourceforge.net>
- ◆QEMU:模拟仿真工具，对内核代码进行调试、跟踪等

7、Linux内核分析方法

SourceInsight

- ◆源代码阅读和编辑工具
- ◆拥有内置的对C/C++，C#和Java等程序的分析。
- ◆能分析源代码并动态维护符号数据库，能自动显示有用的上下文信息。
- ◆到<http://www.sourceInsight.com>上可以下载到它的试用版，试用期估计在30天左右





8、Linux与其他系统的区别

- ◆Linux与MS-DOS
  - 任务调度：MS-DOS是单任务的操作系统，而Linux是多任务的操作系统，用户可以同时运行多个应用程序。
- ◆Linux与Windows
  - ◆Linux是从一个比较成熟的操作系统UNIX发展而来的，是一种开放、免费的操作系统，Windows是封闭的系统，有偿使用；
  - ◆Linux软件丰富，功能完备的GUI
  - ◆稳定性好，硬件适应性强
  - ◆网络功能丰富

8、Linux与其他系统的区别

- ◆Linux与商用UNIX
  - Linux和商用UNIX都基本支持同样的软件、程序设计环境和网络特性，可以说Linux是UNIX的PC版本，Linux在PC机上提供了相当于UNIX工作站的性能。
  - Linux是免费软件，用户可以从网上下载，而商用的UNIX除了软件本身的价格外，用户还需支付文档、售后服务费用
  - Linux拥有GNU软件支持，Linux能够运行GNU计划的大量免费软件，这些软件包括应用程序开发、文字处理、游戏等方面的内容
  - Linux的开发是开放的，任何志愿者都可对开发过程做贡献，而商用UNIX则是由专门的软件公司进行开发的

8、Linux与其他系统的区别

- ◆Linux与自由的UNIX
  - 组织方式：FreeBSD由它的核心团队领导，他们负责原始程序的开发与维护。Linux没有核心团队，来自世界各地的爱好者都可以发布自己的补丁。
  - 发展方向：由于人力有限，FreeBSD的核心团队将主要精力投入在UNIX自身的风格和特点上，不像linux那样对相关硬件支持多，因而在易用上稍微逊色一些。
  - 系统核心功能：二者区别不大，但由于linux庞大的开发队伍，在linux系统上可找到任何需要的功能，许多商用软件才具有的功能在linux上很快就会出现。
  - 性能：据专家分析，FreeBSD在网络性能、软件移植性和系统规范化上略胜一筹，而在硬件支持、磁盘操作等方面Linux略强一些。

9.Linux中的C语言与汇编语言

- ◆Linux源代码中的C语言代码
  - Linux主体是用GNU的C语言编写
  - 从C++中吸收了“inline”和“const”
  - 支持“属性描述符”（attribute）
  - 增加了新的基本数据类型“long long int”用于支持64位CPU
- ◆Linux源代码中的汇编语言代码：两种
  - 完全的汇编代码，以.s作为文件名后缀(可以单独用gcc汇编成目标代码)
  - 嵌入在c程序中的汇编代码

9.Linux中的C语言与汇编语言

- ◆在Linux内核中使用的C语言与通常的C语言有所不同，gcc编译器是一个c/c++编译器，所以在Linux设计中引入了很多C++技术。如：

```
struct Student
{.....
}
```

引用时：  
普通C中：  
Student S;  
gcc中的C: Struct Student S;

9.Linux中的C语言与汇编语言

- ◆与一般的386汇编语言采用intel定义不同，它采用的是AT&T定义的格式。主要差别如下：
  - ◆Intel中多使用大写字母，而这里大多使用小写字母
  - ◆寄存器名前加“%”作为前缀
  - ◆指令的源操作数与目标操作数的顺序与intel的正好相反。AT&T格式中，源在前，目标在后
  - ◆访问内存的指令的操作数大小（即宽度）由操作码名称的最后一个字母决定，用作操作码后缀的字母有（8位），w（16位），l（32位），e.g movb
  - ◆直接操作数要加“\$”作为前缀，intel中不用

9.Linux中的C语言与汇编语言

| 区别项         | AT&T格式                                              | Intel格式                                                           |
|-------------|-----------------------------------------------------|-------------------------------------------------------------------|
| 指令大小写       | 多用小写,如: movw %eax,%ebx                              | 多用大写,如PUSH AX                                                     |
| 寄存器名        | 寄存器名前加%号,如: %eax,%edi                               | 寄存器名前不加任何符号, EAX, EDI                                             |
| 源/目的操作数顺序   | 操作指令 源数 目的数 mov %eax,%ebx;eax--ebx                  | 操作指令目的数 源数 MOV EAX,EBX;EBX--EAX                                   |
| 操作数长度单位表示   | 操作码后最后一字母表示长度单位,其中b(16位),w(32位),l(64位) movb FOO,%al | 操作数前加BYTE PTR, WORD PTR, DWORD PTR或QWORD PTR MOV AL, BYTE PTR FOO |
| 立即数表示       | 立即数前加\$,如: push \$2                                 | 不加任何符号,如PUSH 2                                                    |
| 远跳转/转移/调用指令 | 指令前加字符,如call,jmp                                    | 指令后加FAR 如CALL FAR, JMP FAR                                        |

9.Linux中的C语言与汇编语言

- ◆是一种嵌入在C程序中的汇编
- ◆AT&T行内汇编：在C代码中嵌入汇编代码。在C/C++中提供asm或者 \_\_asm\_\_ 两个关键字，这样使得C代码中嵌入汇编代码非常的简单、方便。
- ◆asm("statements");
  - 例如：asm("pushb %al");asm("popb %al");
- ◆多行汇编可以多个单行汇编，也可以用\_\_asm\_\_修饰符括起，中间插入格式分隔符号，\n表示回车，\t表示跳格，例如：

```
asm("pushl %eax\n\t"
 "movl $0,%eax\n\t"
 "popl %eax");
```

  - 实际上GCC在处理汇编时，把asm括号内的内容按原样拷贝到汇编文件中，所以格式控制字符是必要的。

## 9. Linux中的C语言与汇编语言

- ◆ **扩展行内汇编**：为解决代码中因使用寄存器可能出现的问题，引入了扩展行内汇编。它的基本格式是：

```
asm("statements"
:output_regs //输出寄存器说明
:input_regs //输入寄存器说明;
:clobbered_regs //被改变的寄存器。
```

- 输出和输入寄存器统一按顺序编号，起始是0
- 操作数如何与C代码中的变量结合是个问题。程序员提供具体的指令，而对寄存器的使用只需给出约束条件，具体如何将寄存器与变量结合起来则完全由GCC等负责。

关于嵌入式汇编，请参考<http://gcc.gnu.org/onlinedocs/>上的文档：  
1) 在GCC的文档中有部分

AT&T行内汇编

## 9. Linux中的C语言与汇编语言

```
int a=10, b;
asm(
 "movl %1, %%eax;
 movl %%eax, %0;"
: "r"(b) /* output */
: "r"(a) /* input */
: "%eax" /* clobbered register */
);
```

## 9. Linux中的C语言与汇编语言

- ◆ 上面代码所做的就是用汇编代码把的值赋给b：
  - 1) “b”是输出操作数，用%0来访问，“a”是输入操作数，用%1来访问。
  - 2) “r”是一个constraint，constraint “r”让GCC自己选择一个寄存器去存储变量。
- ◆ 输出部分的constraint前必须要有个“=”，用来说明是一个这是一个输出操作数，并且只写。
  - 3) 有的寄存器名字前面写了两个%，这是用来帮助GCC区分操作数和寄存器。操作数只需要一个%前缀。
  - 4) 在第三个冒号后面的clobbered register部分，%eax说明在内联汇编代码中将要改变eax中的内容，GCC不要用它存储其他值。当这段代码执行结束后，“b”的值将会被改掉，因为它被指定作为输出操作数。换句话说，在“asm”内部对b的改动将影响到asm外面。

## 本章作业

- ◆ 请通过进一步查阅资料了解Linux的发展过程。
- ◆ 解释下列名词：自由软件，GPL，POSIX，GNU
- ◆ Linux内核版本中包含哪些信息？
- ◆ Linux内核的基本组成是什么？各模块之间有什么关系？
- ◆ Linux内核目录结构如何？各目录主要用途有哪些？
- ◆ 什么是Linux的发行版本？写出你知道的发行版本
- ◆ Linux系统有哪些特点？
- ◆ 解释Linux 2.4.18.10内核版本中各数字的含义。